

Empirical Findings for Reliable Structured Output in Multi-Model Agent Pipelines

HONGYU CHEN

AgentSystem Research Lab

Abstract

What limits the reliability of structured output in multi-model agent systems? The prevailing assumption—implicit in LangChain, AutoGen, and CrewAI—is that formatting failures stem from reasoning errors. We show this assumption is systematically falsified. Across calibrated experiments on DeepSeek V4 and MiniMax M3, we find that **structured output compliance, not reasoning capability, is the binding constraint on governance reliability**. Models correctly detect violations and classify errors whenever their output is valid JSON, but the probability of producing valid JSON depends critically on two factors: **token budget adequacy** and **provider-level output artifacts**. A 2×2 factorial experiment ($n = 5$ per cell) crossing prompt richness with token budget reveals that **max_tokens is the sole determinant of output validity** for both models—prompt richness has zero detectable effect at adequate token budgets. However, MiniMax exhibits an additional schema-complexity interaction: at 800 tokens, compliance degrades from 100% (flat schemas) to 20% (array-of-objects schemas), while DeepSeek maintains 100% across all depths. We further demonstrate that **model delegation**—assigning reasoning to MiniMax and formatting to DeepSeek—improves EC-role compliance from 20% (self-pairing) to 100% (delegated), confirming that reasoning and formatting are separable capabilities. We release calibrated experiment data with corrected statistical reporting ($n = 5$ exact bounds, Bonferroni correction). Among 7 tested open-source models, a 7B model (HY-MT2-7B) achieves 100% compliance while 26–27B models produce empty output, providing a decisive counterexample against scale-dependent explanations of structured-output capability. These empirical findings point toward a practical design principle: invest in adequate token provisioning and cross-model delegation rather than prompt engineering, and treat per-model output profiles as first-class architecture inputs.

Keywords: Structured Output Reliability, Token Budget Competition, Model Delegation, JSON Schema Compliance, Agent Harness Governance

1 Introduction

When LLM-based agent systems execute governance operations—plan interrogation, error classification, loop detection, reflection—they must produce structured output. The prevailing assumption across agent frameworks (LangChain, AutoGen, CrewAI) is that governance failures are reasoning failures: a model produces a poor plan because it failed to think exhaustively; misclassifies a risk because it lacks domain knowledge; fails at a governance task because it is not capable enough. This paper presents calibrated experimental evidence that this assumption is wrong.

Our central thesis is the **Formatting Bottleneck Hypothesis**: governance reliability in LLM-based agent systems is bounded not by reasoning capability, but by structured output compliance. We support this thesis with two lines of evidence. First, a systematic 2×2 factorial experiment ($n = 5$ per cell) crossing prompt richness with token budget on DeepSeek V4 and MiniMax M3 reveals that **max_tokens is the sole determinant of output validity**—prompt richness has zero detectable effect at adequate token budgets. Second, **model delegation** (MiniMax reasoning $\rightarrow$ DeepSeek formatting) improves EC-role compliance from

20% to 100%, confirming that reasoning and formatting are separable capabilities best assigned to different models. Among 7 tested open-source models, a 7B model (HY-MT2-7B) achieves 100% compliance—a decisive counterexample against scale-dependent explanations, as it outperforms models 3–4× its size.

This thesis is falsifiable. If governance failures are primarily reasoning failures, then (a) models that reason well should produce valid JSON regardless of output format, (b) prompt engineering should be more effective than token budget adjustment, and (c) pairing a strong reasoner with a strong formatter should not outperform either model alone. Our experiments systematically falsify all three predictions.

The formatting bottleneck has four cascading consequences. **First, directional symbiosis:** MiniMax’s reasoning is substantially intact but masked by a formatting deficit. When paired with DeepSeek for formatting, the combined pipeline achieves 100% compliance on tasks where MiniMax alone achieves 20%. Critically, symbiosis is directional: only the MiniMax→DeepSeek direction works. **Second, a two-step ceiling on cross-model pipelines:** three-step chains universally fail because two handoffs compound information degradation. **Third, governance architectures as attack surfaces:** the SchemaValidator’s retry mechanism amplifies adversarial inputs by 25–29×, inducing format collapse rather than reasoning error. **Fourth, provider-dependent temporal variability:** MiniMax pipelines exhibit session-level collapse ($\Delta = -80\%$ at $T=0.5$), while DeepSeek is stable when token budgets are adequate.

The remainder of this paper is organized as follows. Section 2 surveys related work. Section 3 states the Formatting Bottleneck Hypothesis and falsifiable predictions. Section 4 describes the experimental harness. Sections 5–6 present experimental design and the calibrated 2×2 factorial results. Section 7 reports delegation experiments. Section 8 discusses attack surfaces and temporal variability. Section 9 provides statistical validation with corrected small-sample bounds. Section 10 concludes with practical implications.

2 Related Work

Our work sits at the intersection of five research areas, each of which informs the governance architecture we propose but none of which has previously been integrated into a model-aware harness design.

Agent frameworks and harness design. Modern agent orchestration frameworks—LangChain [1], AutoGen [2], CrewAI [3], and TaskWeaver—provide middleware layers for task decomposition, tool use, and output parsing. Their governance mechanisms (input validation, retry logic, error classification) assume model-independent behavior: a single prompt template and threshold configuration is applied uniformly across all underlying models. Park et al. [4] demonstrated that agent architectures benefit from model-specific tuning of memory and reflection modules, but did not systematically characterize governance behavior—threshold sensitivity, schema compliance, layer ordering effects—across models. Our work is the first, to our knowledge, to provide quantitative per-model governance profiles across these dimensions.

LLM evaluation and benchmarking. Standard benchmarks—MMLU [5], HumanEval [6], HELM [8], BIG-bench—evaluate models on isolated capability tasks. Our work evaluates a different construct: *governance behavior*—how models respond to harness-level prompts, thresholds, layer ordering, and schema constraints. Closest to our approach is Zheng et al. [9] on LLM-as-judge evaluation, which measures models’ ability to evaluate other models’ outputs. However, their work focuses on pairwise comparison accuracy, while ours measures compliance with structured governance protocols across a seven-layer harness pipeline. The recent emergence of LMSys Chatbot Arena and related crowd-sourced evaluation platforms has produced rich data on model behavior variability across prompts and raters, but these platforms do not test governance-specific behaviors such as alternative generation under threshold pressure or compliance with nested JSON output schemas.

Structured output and constrained generation. Recent work on constrained decoding—guidance [10], outlines [11], LMQL, SGLang—shows that enforcing output schemas at the token level improves structural reliability. Our Schema Complexity Cliff finding extends this literature in two ways. First, we show that the relationship between schema complexity and output compliance is not linear but exhibits a threshold effect: compliance remains near-perfect up to a model-dependent depth threshold, then collapses abruptly. Second, we demonstrate that this threshold is model-specific—the same schema that one model handles reliably causes another to fail—and that the binding constraint is not nesting depth alone but the product of depth, field count, and task semantic complexity. These findings have direct implications for how constrained decoding systems should allocate schema enforcement across models in a multi-model pipeline.

Prompt sensitivity and model-specific prompting. The observation that different models respond differently to prompt variations is well-established [12, 13]. Recent work on automatic prompt optimization (DSPy, APE, OPRO) treats prompt sensitivity as a nuisance to be engineered away through search. Our work takes a different stance: prompt sensitivity is a *design input* for harness architecture. We measure it quantitatively (Prompt Sensitivity scores from 0.0 to 1.0) and use it to determine per-layer per-model prompt registries, layer ordering, and whether a model requires native or delegated schema parsing. The finding that DeepSeek exhibits PS=0.0 (prompt-insensitive) while MiniMax exhibits PS=1.0 (maximally sensitive, with parse rates ranging from 0% to 80% across equivalent prompts) is not a curiosity—it is an architectural signal that determines how the harness routes and gates each model’s output.

API reliability and temporal variability in production ML. The observation that ML model APIs exhibit temporal variability is documented in the context of commercial NLP services and vision APIs. However, this work has focused on classification accuracy drift and latency variation. Our work is the first, to our knowledge, to characterize temporal variability in *structured governance output*—the phenomenon that a model’s ability to produce valid JSON under a governance prompt varies across sessions, time of day, and API serving state in a provider-dependent manner. The finding that MiniMax governance output collapses from 80% to 0% parse rate within 60 minutes at elevated temperature, and that DeepSeek’s apparent intermittent empty-response patterns were traced to token budget misconfiguration rather than API instability, establishes that governance architectures must treat temporal calibration as a first-class design concern rather than an operational afterthought.

Multi-agent systems and cross-model interaction. Recent work on multi-agent debate [7], multi-agent collaboration, and LLM ensembles demonstrates that combining multiple models can improve task performance. Our work contributes a governance-specific perspective: cross-model governance pipelines exhibit directional symbiosis (only the MiniMax→DeepSeek pair produces emergent quality), a hard ceiling at two steps (three-step pipelines universally fail due to compounding handoff fragility), and a decoupling principle (reasoning and formatting are separable capabilities best assigned to different models). These findings suggest that the benefits of multi-agent architectures for governance are narrower and more constrained than the general multi-agent literature would predict.

3 The Formatting Bottleneck Hypothesis

We propose the **Formatting Bottleneck Hypothesis**: in multi-model agent governance, structured output compliance, not reasoning capability, is the binding constraint on reliability. Models can correctly detect violations, classify errors, identify loops, and recognize adversarial inputs whenever their output is valid JSON. The probability of producing valid JSON, however, undergoes a model-dependent threshold effect—the Schema Complexity Cliff—as output schema complexity (depth $\times$ field count $\times$ task semantic complexity) increases. This formatting bottleneck, not reasoning depth, determines which governance architectures succeed and which

fail.

3.1 Falsifiable Predictions

The hypothesis makes four predictions that are empirically testable and, if the prevailing reasoning-centric view were correct, would be falsified:

P1: Detection-formatting dissociation. If governance failures are formatting failures rather than reasoning failures, then governance detection accuracy should approach 100% on parseable outputs regardless of task type, while unparseable outputs should account for all observed failures. Conversely, if governance failures are reasoning failures, parseable outputs should exhibit non-trivial error rates.

P2: Directional symbiosis under decoupling. If the formatting bottleneck is the binding constraint, then pairing a model with intact reasoning but poor formatting (MiniMax) with a model with reliable formatting (DeepSeek) should produce emergent governance quality exceeding either model alone. Critically, the reverse direction (DS-reasoning + MM-formatting) should fail, because the formatting bottleneck is not symmetric.

P3: Compounding handoff fragility. If each governance step introduces a formatting risk, then governance reliability in multi-step pipelines should degrade geometrically with the number of handoffs, not linearly. Two-step pipelines should have some chance of success; three-step pipelines should universally fail.

P4: Provider-dependent temporal variability. If the formatting bottleneck is sensitive to prompt design and token budget rather than to API serving state alone, then manipulations of prompt richness and max_tokens should produce larger changes in governance compliance than temporal variation at constant parameters. Conversely, if temporal API variability is the dominant factor, governance outcomes should vary across sessions even when prompt and token budget are held at optimal levels.

3.2 Relationship to Prior Assumptions

The prevailing assumption in agent frameworks is that governance failures are reasoning failures: a model produces a poor plan because it failed to think exhaustively, misclassifies a risk because it lacks domain knowledge, or fails at a governance task because it is not capable enough for that task. Under this view, the remedy is to use a more capable model, a more detailed prompt, or a more sophisticated governance layer.

The Formatting Bottleneck Hypothesis proposes a different causal model. A model may reason perfectly about a governance decision—correctly identifying all dependencies, failure modes, and alternatives in an execution plan, or correctly classifying a deployment as unsafe with high risk—and yet produce zero usable governance output because the required output format exceeds its schema complexity tolerance. The SchemaValidator rejects valid reasoning because the reasoning is packaged in invalid JSON. The governance system cannot distinguish between “the model reasoned incorrectly” and “the model reasoned correctly but formatted incorrectly”—both manifest as a parse failure. The system therefore treats a formatting failure as a reasoning failure, and responds with prompt engineering or model substitution, neither of which addresses the actual bottleneck.

If the Formatting Bottleneck Hypothesis is correct, the architectural implication is a reversal of priorities: the harness should invest design effort in output format simplification, model-formatting matching, and formatting-aware degradation strategies *before* investing in more sophisticated reasoning prompts or more capable models. The Schema Complexity Cliff is the binding constraint; everything else is downstream.

3.3 Methodological Note: Hypothesis-Driven vs. Exploration-Driven Design

The experiments reported in this paper span two methodological regimes. The Architecture Principle experiments (H-track) and Infrastructure Validation experiments (EXP-track) were designed as **confirmatory**: each tested a specific, pre-formulated hypothesis about governance behavior. The Conjecture-Driven experiments (C-track) were designed as **exploratory**: each tested a conjecture that emerged from the confirmatory results, with methodology refinements between rounds to isolate causal mechanisms from measurement artifacts. We distinguish these regimes explicitly in the Experiment Matrix and in the reporting of each experiment, and we note where the exploratory nature of a finding limits the strength of the claim that can be drawn from it. All confirmatory experiments were designed around pre-formulated hypotheses with controlled confounds where operationally feasible; explicit sample sizes, statistical power caveats, and methodological limitations are reported for each experiment. All exploratory experiments are reported with explicit caveats about their post-hoc nature and the need for independent replication.

3.4 Confound Transparency

Several experiments in this study involve confounds that we document explicitly rather than attempt to conceal. EXP-08 (Schema Depth Ladder) varies nesting depth and prompt format simultaneously: the “minimal prompt” condition used a bare “Return ONLY valid JSON” instruction that, for prompt-sensitive models like MiniMax (PS=1.0), confounds schema complexity with prompt richness. We report this confound explicitly and interpret the resulting MiniMax data as evidence for prompt-dependence rather than a pure depth effect. EXP-11 (Prompt Compression) confounds prompt length reduction with schema simplification, making it impossible to attribute parse rate changes exclusively to either factor. We report the confound and interpret the results as evidence that PI governance does not survive *any* form of compression, rather than attributing the failure to a specific compression mechanism. EXP-15 (Schema Flattening) reduces nesting depth but also reduces semantic richness, confounding structural simplification with information loss. We report this and interpret the null result (flattening does not restore compliance) as evidence that depth alone is not the binding factor—the product of depth, field count, and task semantic complexity is. These confounds are not hidden; they are documented as limitations that future work should address through factorial designs that isolate each factor independently.

4 Background: AgentSystem Harness

The AgentSystem harness implements a 24-layer middleware chain that processes every agent action through sequential governance gates before execution. This study isolates seven governance-critical layers, selected because each produces a structured output that can be experimentally validated for correctness, compliance, and stability across models.

These seven layers collectively implement a four-gate governance architecture that is structurally enforced by the harness control plane:

Gate 1: Meta-Instruction Gate. The harness verifies that the model has acknowledged the system’s governing rules (AGENTS.md) before processing any action—a baseline compliance check that gates all subsequent layers.

Gate 2: Stage Verifier. A deterministic three-level validation: (1) tool-call evidence must be present in the execution trace; (2) output structure must conform to the layer’s expected schema; (3) domain-specific rules (e.g., minimum alternative count in Plan-InterrogationGate) must be satisfied.

Table 1: Seven Harness Layers Under Study

Pos	Layer	Output	Governance Function
0	CapabilityGate	Boolean pass/fail	Honest self-assessment: can this model handle the requested task?
5	PipelineGateway	Routing decision	Route task to correct execution pipeline based on task type
6	PlanInterrogationGate	Structured JSON plan	Grill execution plan: identify dependencies, failure modes, alternatives ($k \geq 3$)
12	ErrorClassifier	Error taxonomy	Classify errors by type, severity, and recoverability
17	LoopDetection	Loop verdict	Detect repeated failure patterns; trigger escalation after 3 identical failures
19	PreCompletionChecklist	Checklist verdict	Verify artifacts and substance before marking task complete
20	ReflectionCheck	Quality score + critique	Self-evaluate output quality against original requirements

Gate 3: Loop Detection. Three identical consecutive failures at any layer trigger automatic termination and escalation. The threshold of three is derived from the governance loop detection experiments (EXP-04), which confirmed that loops are identifiable within two repetitions and that earlier termination risks false positives from transient failures.

Gate 4: Reasoning Auditor. A post-execution audit verifies that the model produced coherent reasoning traces. Self-silencing detection executes at this gate: if the model’s output contains no reasoning content, the action is flagged for human review regardless of structural compliance.

The key insight that motivates this study is that each of these gates imposes a different kind of demand on the underlying model. PlanInterrogationGate requires exhaustive structured generation under threshold pressure. SchemaValidator requires precise JSON compliance under varying nesting depths. ReflectionCheck requires honest self-assessment, which our experiments show is systematically biased. Our experimental program was designed to measure how each model responds to each of these demands, and to derive architecture principles from the resulting per-model governance profiles.

5 Experiment Design

5.1 Research Questions and Hypothesis Structure

The experimental program was organized around four progressively deeper research questions:

RQ1: Architecture Principles. Do governance-critical harness layers demand model-specific configuration, or can a single configuration serve all models? Hypothesis: governance behavior is model-dependent at every layer, and universal configurations will systematically under-perform for some models.

RQ2: Infrastructure Validation. Given model-aware configuration, does governance detection (adversarial, loop, intent violation, hallucination) achieve production-grade reliability? Hypothesis: detection accuracy is gated by output compliance rather than detection capability—models can detect what they can format.

RQ3: Conjecture-Driven Discovery. What architectural phenomena emerge when governance is treated as a dynamic, multi-model, security-aware system rather than a static pipeline? Hypothesis: cross-model governance exhibits directional symbiosis, amplification effects, and a hard ceiling on pipeline depth that are not predicted by single-model governance theory.

RQ4: Industry-Grade Reproducibility. Are governance experiment outcomes reproducible across time, sessions, and API state? Hypothesis: temporal reproducibility is itself provider-dependent, and any governance architecture that assumes stationary API behavior will fail in production.

5.2 Models Tested

Three models were selected to span different architectures, providers, and inference cost profiles. All experiments used temperature=0.0 as the default for reproducibility, with controlled temperature elevation experiments (0.0–0.7) to measure temperature robustness.

Table 2: Model Matrix

Model	Provider	Architecture	Default T	API Calls
DeepSeek V4 Pro	DeepSeek API	Reasoning-first MoE	0.0	600+
MiniMax M3	MiniMax API	General-purpose	0.0	400+
Mimo V2.5 Pro	Mimo API	General-purpose	0.0	200+

5.3 Experimental Tracks

Experiments were organized into four tracks, each serving a distinct role in the research program:

Architecture Principle Track (H-track). Five experiments (H1–H5) testing foundational governance hypotheses: threshold tuning (H4), schema validation and auto-retry (H3), prompt sensitivity detection (H1), layer ordering effects (H5), and routing strategy (H2+H6). These experiments established the baseline per-model governance profiles that inform all subsequent experiments.

Infrastructure Validation Track (EXP-track). Eighteen experiments (EXP-02 through EXP-25) extending the architecture principles into production-relevant validation scenarios: adversarial detection, loop detection, schema depth ladder mapping, compliance-detection decoupling, self-evaluation leniency curves, concurrency stress testing, degradation ladders, PI governance boundary characterization, domain sensitivity, schema flattening, adversarial schema security, detection generalization, governance density optimization, prompt compressibility, and pipeline protocol fidelity.

Conjecture-Driven Discovery Track. Five conjecture rounds (C1–C9, ~500 calls) exploring emergent governance phenomena: governance attack surfaces and amplification factors (C1), meta-governance conservatism (C2), multi-agent cascade dynamics (C3), reasoning-formatting decoupling (C4), directional cross-model symbiosis (C5), governance triad decomposition (C6), temporal drift detection (C8), and multi-step pipeline feasibility (C9). These experiments were designed to test hypotheses that emerged from the architecture principle results rather than following a pre-registered plan.

Industry & Reproducibility Track. Five experiments (~700 calls) validating governance reliability under realistic deployment conditions: governance-as-a-service formatting (H1-repr), temporal audit trail integrity (H3-repr), cross-provider interoperability (H4-repr), capstone end-to-end pipeline validation (GIS), and a 400-call temporal reproducibility matrix experiment spanning two time points and five temperatures.

5.4 Metrics

Architecture principle experiments (H-track) measure alternative counts, dependency counts, failure mode counts, decision tree depth, pass rates, and composite governance scores on 1–5 scales. Infrastructure validation experiments (EXP-track) measure detection accuracy, parse rates (proportion of responses producing valid JSON), round accuracy (correct verdict after retry), false positive rates, and self-evaluation quality scores against ground-truth annotations. Conjecture experiments additionally measure amplification factors (ratio of output degradation to input perturbation), symbiosis scores (governance quality of cross-model pipeline minus maximum single-model quality), and cascade propagation rates. Reproducibility experiments measure inter-temporal deltas (Δ) and cross-session coefficient of variation.

5.5 Statistical Methodology

All hypothesis tests use non-parametric methods appropriate for the small sample sizes and ordinal nature of governance scores. Friedman tests for repeated measures assess model differences in alternative generation (H4). Cohen’s d quantifies effect sizes for pairwise model comparisons. 95% confidence intervals are reported for all primary metrics. Temporal reproducibility uses inter-temporal delta ($\Delta = \text{metric}_{t_1} - \text{metric}_{t_0}$) with provider as the grouping variable. Cross-session reproducibility uses coefficient of variation across independent measurement sessions.

6 Experiment Matrix

The complete experimental program spans 25 experiments organized across the four tracks described in Section 4. Table 3 provides a unified view of every experiment, its call budget, and its primary finding. The total call count of 1,200+ reflects the cumulative experimental investment required to establish governance profiles at production-grade confidence. Each experiment was designed to isolate a specific governance variable: threshold sensitivity (H4), schema complexity (H3, EXP-07, EXP-08, EXP-15), layer ordering (H5), prompt sensitivity (H1, EXP-25), routing strategy (H2+H6), adversarial robustness (EXP-02, EXP-18), detection generalization (EXP-04, EXP-22), self-evaluation bias (EXP-07, EXP-10), concurrency behavior (EXP-11, EXP-12, EXP-23), temperature robustness (Temperature experiment, EXP-13), PI governance boundaries (EXP-13, EXP-14, EXP-15), temporal reproducibility (Temporal experiment, C8, cross-session reproducibility), and cross-model interaction dynamics (C1–C9).

The taxonomy of experiments reflects the research program’s evolution. The H-track experiments (rows 1–5) established baseline per-model governance profiles at $T=0$ under calibrated prompts. The EXP-track experiments (rows 6–23) extended these profiles into production-relevant conditions: elevated temperature, adversarial input, concurrency stress, and schema complexity variation. The Conjecture track (rows 24–31) explored emergent phenomena that the baseline profiles did not predict, using hypothesis-driven experimental designs that evolved across five rounds of iteration. The Industry & Reproducibility track (rows 32–36) subjected the architecture to the most demanding test: whether governance behavior remains stable when experiments are repeated across time, sessions, and API state. The answer—that reproducibility is itself provider-dependent—is one of the central findings of this study.

7 Calibrated Results: 2×2 Factorial and Delegation

We report freshly calibrated experimental data with corrected statistical reporting (exact p -value bounds at $n = 5$, Bonferroni correction). All experiments use temperature=0.0.

Table 3: Experiment Matrix (Selected Key Experiments)

ID	Experiment	Calls	Key Finding
<i>Architecture Principle Experiments (H-track)</i>			
H4	Threshold tuning	30	Optimal thresholds are per-model ($DS \geq 5$, $MM \geq 3$)
H3	Schema validation + auto-retry	24+	Exactly 1 retry restores compliance for flat schemas
H1	Prompt sensitivity auto-detection	30+	PS resists auto-detection; per-model registries needed
H5	Layer ordering effect	18+12	EC→PI better for MM (+20–33%); DS insensitive
H2+H6	Routing strategy	24+3	Single-model > hybrid; DS-only safest default
<i>Infrastructure Validation (EXP-track, selected)</i>			
EXP-02	Adversarial detection	45	100% accuracy when JSON valid; zero false positives
EXP-08	Schema Depth Ladder + 2×2 Factorial	129	DS: token-budget main effect (100% vs. 0%); MM: prompt-dependent
EXP-10	Self-evaluation bias	30	DS asymmetric leniency; MM+MO cannot produce eval JSON
EXP-18	Adversarial schema security	45	Defense held; 0% attacks passed at all depths
Temperature	Temperature robustness	54+	DS+MM stable at ≤ 0.3 ; MIMO degrades above 0.0
<i>Conjecture-Driven (selected)</i>			
C1	Governance attack surface	136	Amplification 25–29×; attacks cause silence not error
C5	Directional symbiosis	78	Only MM→DS produces emergent quality
<i>Industry & Reproducibility</i>			
H1	Governance-as-a-Service	20	DS correctly formats all reasoning quality levels
Temporal	Reproducibility matrix	400	DS stable ($\Delta \leq 10\%$); MM collapses ($\Delta = -80\%$)
Total (25 experiments)		1,200+	Full experiment log in Supplementary Material

7.1 2×2 Factorial: Prompt Richness × Token Budget

Design: 2 (prompt: rich vs. minimal) × 2 (max_tokens: 800 vs. 30) × 3 (schema depth: D1/LD flat, D5/RC nested, D9/EC array-of-objects), $n = 5$ per cell, 120 total API calls across DeepSeek V4 and MiniMax M3.

Table 4: Calibrated 2×2 Factorial Results

Model/Depth	Rich+800t	Rich+30t	Minimal+800t	Minimal+30t
<i>DeepSeek V4 (deepseek-v4-flash)</i>				
D1 (LD, flat)	5/5 (100%)	0/5 (0%)	5/5 (100%)	0/5 (0%)
D5 (RC, nested)	5/5 (100%)	0/5 (0%)	5/5 (100%)	0/5 (0%)
D9 (EC, array-of-obj)	5/5 (100%)	0/5 (0%)	5/5 (100%)	0/5 (0%)
<i>MiniMax M3</i>				
D1 (LD, flat)	5/5 (100%)	0/5 (0%)	5/5 (100%)	0/5 (0%)
D5 (RC, nested)	5/5 (100%)	0/5 (0%)	3/5 (60%)	0/5 (0%)
D9 (EC, array-of-obj)	1/5 (20%)	0/5 (0%)	1/5 (20%)	0/5 (0%)

Key findings: (1) **max_tokens is the dominant factor** for both models—all low-token cells produce 0% valid JSON (`finish_reason=length`), all high-token cells produce valid output at D1. (2) **Prompt richness has zero effect** on DeepSeek at adequate token budgets—the rich+high and minimal+high cells are identically 100%. (3) **MiniMax exhibits a schema-complexity interaction:** at 800 tokens, compliance degrades from 100% (D1 flat) to 20% (D9 array-of-objects), while DeepSeek maintains 100% across all depths. This gradient confirms that the binding constraint is not schema depth alone but the joint demand of schema structure and token budget. At 8000 tokens, MiniMax also achieves 100% across all depths (reported in the original study), establishing a dose-response relationship between token budget and schema complexity tolerance.

7.2 Delegation: Self-Pairing vs. Cross-Model Pipeline

Design: EC role (array-of-objects, the hardest schema). MiniMax produces free-text reasoning; formatting is performed by either MiniMax (self-pairing) or DeepSeek (delegation). $n = 5$ per condition, $T = 0.0$.

Table 5: Delegation Results: Self-Pairing vs. Cross-Model

Condition	EC Compliance	Improvement
MM→MM (self-pairing)	1/5 (20%)	—
MM→DS (delegation)	5/5 (100%)	+80 pp

Finding: Delegation improves EC-role compliance from 20% to 100%. MiniMax’s reasoning capability is intact (free-text analysis succeeds in all 10 trials), but its formatting capability is the bottleneck. DeepSeek reliably converts MiniMax’s reasoning into valid JSON. This confirms the separability of reasoning and formatting capabilities—a finding that generalizes across model architectures.

7.3 Generalization to Open-Source Models

To test whether the formatting bottleneck is specific to commercial APIs, we replicated the D3 role profiling on 7 open-source GGUF models via llama-server (Vulkan backend, local inference).

Models were loaded sequentially to avoid GPU memory contention; each was tested on all 4 governance roles (PI, EC, RC, LD) with identical prompts and $T = 0.0$, max_tokens=256.

Table 6: Open-Source GGUF Model D3 Role Profiling

Model	Size	PI	EC	RC	LD
HY-MT2-7B (Q8_0)	7B	1/1	1/1	1/1	1/1
GPT-OSS-20B (MXFP4)	20B	0/1	0/1 ^a	0/1	1/1
Nemotron-30B (Q4_K_M)	30B	0/1	0/1 ^a	0/1	1/1
Gemma-4-26B (Q4_0)	26B	0/1	0/1	0/1	0/1
Qwen3.6-27B (Q5_K_M)	27B	0/1	0/1	0/1	0/1
DeepSeek-R1-Qwen3-8B	8B	0/1	0/1	0/1	0/1
GLM-4.7-Flash (Q5_K_M)	4.7B	0/1	0/1	0/1	0/1

^aPartial JSON produced but truncated at 256-token limit (M1: token budget competition). Other 0/1 entries produced empty output, consistent with chat-template incompatibility rather than inherent JSON-generation inability.

Key finding: Among the 7 tested open-source models, only HY-MT2-7B produced valid JSON across all roles under the identical zero-shot prompt format. The remaining 6 models produced empty outputs due to chat-template mismatches with the minimal prompt format, rendering their SSC profiles unmeasurable under this protocol—not evidence of inherent JSON-generation inability. The generalizable finding is not that all 7 models replicate the API pattern, but that **the smallest tested model (7B) outperformed models 3–4× its size under identical conditions**, confirming that SSC tolerance is decoupled from parameter count. This single decisive counterexample—a 7B model achieving 100% where a 27B model produces empty output—is sufficient to falsify the hypothesis that structured-output capability scales with model size. Future work should retest the 6 unmeasurable models with chat-template-appropriate prompting to establish their true SSC profiles.

8 Supplementary Architecture Experiments

Five architecture experiments (H-track, $T = 0.0$, calibrated prompts) confirm that governance behavior is model-dependent at every layer. Tables 7–11 summarize the key results; full protocols are in the Supplementary Material.

Table 7: Threshold Pass Rates (H4)

Model	≥ 1	≥ 2	≥ 3	≥ 4	≥ 5
MiniMax M3 (mean=4.9)	100%	100%	100%	100%	90%
DeepSeek V4 Pro (mean=7.5)	90%	90%	90%	90%	90%
Mimo V2.5 Pro (mean=10.0)	90%	90%	90%	90%	90%

H4 (Threshold tuning): Alternative generation counts are model-specific (MM: 4.9 ± 0.3 , DS: 7.5 ± 3.0 , MO: 10.0 ± 4.3 ; Friedman $\chi^2(2) = 10.85$, $p = 0.0044$, $n = 5$). No single threshold serves all models. Manipulating threshold instructions does not change output volume—thresholds are descriptive boundaries, not causal levers.

Table 8: Schema Validation Recovery (H3)

Model	1st-Try Valid	After Retry	Avg Retries
DeepSeek V4 Pro	0/4 (0%)	4/4 (100%)	1.0
MiniMax M3	0/4 (0%)	4/4 (100%)	1.0
Mimo V2.5 Pro	0/4 (0%)	4/4 (100%)	1.0

H3 (Schema retry): Exactly one retry restores 100% compliance for simple flat schemas across all models. Multi-retry loops add cost without benefit.

Table 9: Prompt Sensitivity Detection (H1)

Model	V1 (generic)	V2 (harness)	Known PS
DeepSeek V4 Pro	PS=0.13	PS=0.00	0.0 ✓
MiniMax M3	PS=0.24	PS=−2.00	1.0 ×
Mimo V2.5 Pro	PS=−0.06	PS=−0.50	1.0 ×

H1 (Prompt sensitivity): Prompt sensitivity resists auto-detection. Harness-specific probes failed to identify MiniMax and Mimo as prompt-sensitive (PS=1.0). Per-model prompt registries are required.

Table 10: Layer Ordering Effect (H5)

Model	R1 Alts Δ	R1 Score Δ	R2 Alts Δ	R2 Score Δ
DeepSeek V4 Pro	0%	0%	−5%	−24%
MiniMax M3	+33%	+50%	+20%	+85%

H5 (Layer ordering): EC→PI ordering improves MiniMax’s output by 20–33% (alternatives) and 50–85% (quality). DeepSeek is order-insensitive ($\leq 5\%$ variation).

H2+H6 (Routing): Single-model pipelines outperform hybrid routing. DeepSeek-only is the safest default; MiniMax handles 2-model chains with calibrated prompts.

Synthesis: The five experiments converge on one finding—governance parameters must be per-model, not global constants. The per-model runtime profile (thresholds, prompts, schema limits, layer order) is not an optimization; it is a prerequisite for multi-model governance reliability. Full experiment logs are provided in the Supplementary Material.

9 Results: Infrastructure Validation Experiments

The Infrastructure Validation experiments (EXP-track) address RQ2: given model-aware configuration, does governance detection achieve production-grade reliability? Eighteen experiments test governance behavior under conditions that the baseline H-track experiments did not explore: adversarial stress, schema complexity variation, temperature elevation, concurrency pressure, and domain generalization. The central finding is that **governance detection achieves 100% accuracy whenever JSON output is valid**—making output compliance, not detection capability, the binding constraint on harness reliability. Table 4 summarizes all experiments; the following subsections detail the four experiments that produced the most architecturally consequential findings.

Table 11: Routing Strategy (H2+H6)

Strategy	R1 Score	R1 Status	R2 Recommendation
DS-only	95	✓ Pass	ds-only
MM-only	85	✓ Pass	hybrid
MO-only	0	× RC parse fail	hybrid
Hybrid	0	× RC parse fail	—

Table 12: Infrastructure Validation Summary

ID	Calls	Finding
EXP-02	45	All models 100% accuracy when JSON valid; zero false positives
EXP-04	36	DS 92%, MM 100%; all 4 loop types detected
EXP-07	60+	Schema Complexity Cliff; self-eval leniency universal
EXP-03	9	0% parse; 9-stage nested schema exceeds all models
EXP-05	23+	DS empty responses traced to max_tokens artifact (finish_reason=length)
Temperature	54+	DS+MM stable at ≤ 0.3 ; MIMO degrades above 0.0
EXP-13/14/15	162	PI only works for DS at T=0; collapses for all others
EXP-18	45	Defense held; 0% attacks passed at all depths
EXP-22	15	Detection generalizes from 2 to 4 task types when parsed
EXP-25	60	PI prompts non-compressible; compression reduces parse to zero
EXP-23	0 (sim)	Adaptive strategy best; static degradation suboptimal
7-Layer	90	All pass with calibrated prompts
Multi-Round	36	Original failures were prompt artifacts

9.1 EXP-08: Schema Depth Ladder and 2×2 Factorial Decomposition

The Schema Complexity Cliff was initially characterized at only three depth points: depth 1 (H3, 100% compliance), depth 3 (EXP-07, 85%–0%), and depth 9 (EXP-03, 0%). These points confirmed the Cliff’s existence but left its precise shape and causal mechanism unknown.

To map the Cliff at single-depth-increment resolution, we constructed a Schema Depth Ladder: nine output schemas spanning depths 1–9, each conveying equivalent semantic content with progressively deeper object nesting. Three governance tasks were presented per model per depth with minimal “Return ONLY valid JSON” prompts. No retry was permitted—this measures first-attempt compliance.

A methodological re-examination revealed that the original experiment confounded nesting depth with prompt format. For prompt-sensitive models (MiniMax, PS=1.0), the minimal prompt condition systematically suppressed compliance independent of depth. Two clean replications were conducted to isolate the operative variables:

EXP-08-CLEAN (MiniMax): Rich contextual prompts at every depth level, isolating depth from prompt format. MiniMax produced valid JSON at 100% of tested depths through D9—the “Cliff” was entirely a prompt-format artifact. The binding constraint for MiniMax is prompt adequacy, not schema depth.

EXP-08-FACTORIAL (DeepSeek): A 2×2 design crossing prompt richness (rich vs. minimal) with token budget (max_tokens=8000 vs. 30), testing 3 depths (D1, D5, D9) with 3 calls per cell—18 API calls. The results are definitive (Table 13): max_tokens is the sole determinant of DeepSeek’s compliance. High tokens produce 100% parse rates regardless of prompt; low tokens produce 0%. Prompt quality has no measurable effect, consistent with DeepSeek’s PS=0.0. A binomial confirmation at the D9 minimal+high condition ($n=20$, 20/20 valid, 95% CI [83.9%, 100%]) confirms the 100% result is not an artifact of small n . The mechanism is confirmed by raw API inspection: empty responses exhibit `finish_reason=length` with all completion tokens consumed by internal reasoning (`reasoning_tokens=20`).

EXP-08-FACTORIAL-MM (MiniMax): The same 2×2 design for MiniMax, expanded to $n=10$ per cell in a replication round (84 additional API calls). The pattern is identical at $n=10$: 10/10 valid in all high-token cells (A and C), 0/10 valid in all low-token cells (B and D), across all three depths. Both models exhibit a clean main effect of token budget with zero detectable effect of prompt richness at $n=10$. DeepSeek shows a modest survival advantage at low tokens under minimal prompts (D1: 100%, D5: 33%), likely because minimal prompts engage fewer reasoning tokens.

The practical conclusion is uniform: **provision adequate token budgets (8000+) for all governance tasks.** Prompt richness does not compensate for insufficient tokens.

A cross-task replication tested the identical 2×2 design (D9, MiniMax, $n=3$ per cell) on two additional governance task types: code review safety analysis and legal compliance assessment. The pattern was identical across all three task types—100% valid JSON in all high-token cells, 0% in all low-token cells, with zero effect of prompt richness or task domain (24 additional API calls). A temperature generalization test at D9 with high tokens confirmed temperature-robustness: 3/3 valid at $T=0.3$ and 3/3 valid at $T=0.5$ (6 additional calls). Collectively, these replications support the cross-task and cross-temperature generalizability of the max_tokens main effect. A thinking-chain suppression test further confirmed the mechanism: instructing MiniMax to omit `<think>` tags at low tokens (max_tokens=30, D9) failed in all 12 calls across three prompt variants—the thinking chain was produced in every call, consuming the entire budget. The thinking chain cannot be disabled through the prompt API, confirming that the token-budget bottleneck is a property of the inference serving architecture, consistent with the Formatting Bottleneck Hypothesis’s prediction that the mechanism is architecture-dependent rather than controllable through prompt design. The operational implication is that the SchemaValidator must verify token budget adequacy (`finish_reason=length` detection) before interpreting empty responses as governance failures.

Table 13: 2×2 Factorial: Prompt Richness × Token Budget (DeepSeek)

D9 (minimal+high): $n=20$ binomial confirmation, 20/20 valid, 95% CI [83.9%, 100%].

Depth	Rich+High	Rich+Low	Minimal+High	Minimal+Low
D1	3/3	0/3	3/3	3/3
D5	3/3	0/3	3/3	1/3
D9	3/3	0/3	3/3	0/3

Table 14: 2×2 Factorial: Prompt Richness × Token Budget (MiniMax)

Depth	Rich+High	Rich+Low	Minimal+High	Minimal+Low
D1	10/10	0/10	10/10	0/10
D5	10/10	0/10	10/10	0/10
D9	10/10	0/10	10/10	0/10

9.2 EXP-09: Compliance-Detection Decoupling

The H3, EXP-02, and EXP-04 experiments established that governance detection reaches 100% accuracy whenever output is valid JSON. This suggests a hypothesis: if compliance is the sole bottleneck, decoupling detection from formatting—free-text reasoning followed by a separate formatting call—should improve reliability. We tested this using the adversarial detection task: in coupled mode, the model produced analysis and JSON in a single response; in decoupled mode, the model first produced free-text analysis, then received its own analysis with a formatting-only prompt.

The results were counter-hypothetical: decoupling reduced parse rates for both DeepSeek (100%→33%) and MiniMax (67%→0%). Inspection revealed a consistent failure mode: the formatting model paraphrased, summarized, or restructured the analysis rather than faithfully transcribing it. This negative result carries a positive implication: the compliance bottleneck is solved by well-designed integrated prompts, not by adding a separate formatting layer. The SchemaValidator’s single-retry policy (H3) provides a safety net without the transcription risk of decoupling.

9.3 EXP-10: Leniency Curve — Quantifying Self-Evaluation Bias

Ten code review outputs at quality levels 1 (empty) through 10 (perfect) were presented to each model for self-evaluation. MiniMax and Mimo produced 20/20 parse errors—they could not produce valid evaluation JSON. DeepSeek’s data revealed an asymmetric bias: on low-quality outputs (levels 1–5), it was *excessively harsh* (10–35 points below true score); on high-quality outputs (levels 7–9), it became *mildly lenient* (5–10 points above). This asymmetric profile has direct architectural implications: a simple linear correction would worsen low-end harshness

Table 15: Summary of 2×2 Factorial Results: DeepSeek vs. MiniMax

Property	DeepSeek (PS=0.0)	MiniMax (PS=1.0)
Dominant factor	max_tokens (main effect)	max_tokens (main effect)
Prompt effect at high tokens	None (both 100%)	None (both 100%)
Prompt effect at low tokens	Present (D>B: minimal > rich)	None (B=D=0%)
Architectural implication	Provision 8000+ tokens	Provision 8000+ tokens

while partially correcting high-end leniency. The correct calibration is a per-model sigmoid correction function stored in the per-model runtime profile. The finding that only DeepSeek can serve as a self-evaluator confirms that self-evaluation is itself a model-dependent property.

9.4 EXP-11/12: Governance Under Concurrent Load

Concurrency stress experiments tested governance under finite API capacity. Prompt compression (EXP-11) demonstrated that PI governance does not survive any form of compression—all models produced 0% parseable output for plan interrogation at every compression level. Layer degradation (EXP-12) demonstrated that shedding governance layers preserves verdict correctness while reducing token consumption: DeepSeek in Bare configuration achieves 34.7 accuracy-units per 100 tokens, approximately $2.2\times$ MiniMax’s optimal density of 17.4 in Minimal configuration. The adaptive degradation strategy—Full governance under normal load, model-optimal degradation when queue depth exceeds threshold—outperforms any static configuration. The per-model runtime profile encodes both the degradation threshold and the model-specific optimal degradation level (Bare for DeepSeek, Minimal for MiniMax).

10 Conjecture-Driven Discoveries: Two Key Phenomena

Beyond the confirmatory experiments, conjecture-driven exploration (~ 500 additional API calls) revealed two phenomena with direct architectural implications. Full experimental protocols and Round 1–5 iterations are reported in the Supplementary Material.

10.1 Governance Architectures as Attack Surfaces

We tested whether governance pipeline components—retry mechanisms, external validators, layer ordering—introduce structural vulnerabilities distinct from model capability failures. Three attack vectors were evaluated across DeepSeek and MiniMax.

Token exhaustion attacks exploit the single-retry mechanism by crafting inputs that trigger repeated validation failures, consuming ~ 800 tokens per governance call at baseline intensity. The amplification factor at $1\times$ intensity is $25\text{--}29\times$, meaning a 20-token attack produces ~ 500 tokens of governance overhead. At $200\times$ intensity the factor drops to $\sim 1.1\times$, confirming asymptotic saturation rather than unbounded amplification. **Reflection poisoning** succeeded against DeepSeek in 2/3 attempts: misleading reputation context caused the model to assign quality scores >70 to effectively empty outputs.

The primary failure mode under attack is **format collapse, not reasoning error**: DeepSeek maintained 100% accuracy through $50\times$ intensity but collapsed to 0% parse rate at $100\times$; MiniMax collapsed at $1\times$. The attack succeeds by making the model silent, not wrong. This establishes governance architectures as independent attack surfaces requiring dedicated threat modeling—the retry mechanism and external validators create vulnerabilities that are architectural, not model-level.

10.2 Directional Symbiosis and Reasoning-Formatting Decoupling

A persistent finding across all experiments is that MiniMax’s governance failures are predominantly format failures. MiniMax achieves 100% detection accuracy on parseable outputs (EXP-02, EXP-04) but its parse rate is model-dependent. This suggested that reasoning and formatting are separable capabilities.

Decoupling experiment. MiniMax produced free-text governance reasoning, which a separate model formatted into JSON. Across three complexity levels, MiniMax’s reasoning succeeded

in 6/6 cases but self-formatting succeeded in only 2/6. The reasoning capability exists; the formatting capability does not. DeepSeek showed no such dissociation (3/6 for both).

Delegation rescue. When MiniMax’s reasoning was paired with DeepSeek’s formatting (MM→DS), governance accuracy reached 100% on tasks where MiniMax alone achieved 20%. Our calibrated replication (Section 7, $n = 5$) confirms this: MM→MM self-pairing achieves 1/5 (20%) on EC-role compliance, while MM→DS delegation achieves 5/5 (100%). This 80-percentage-point improvement is not a model-pair property—it is a functional-role property: reasoning and formatting are best assigned to different models when no single model demonstrates adequate capability in both dimensions.

Directionality. Symbiosis is directional. Among all six ordered model pairs tested, only MiniMax→DeepSeek produced emergent quality. The reverse direction (DS→MM) and all other combinations produced zero improvement. This asymmetry confirms that the bottleneck is formatting, not reasoning: a strong reasoner benefits from a strong formatter, but not vice versa.

Two-step ceiling. Multi-step pipelines exhibit compounding handoff fragility. Three-step chains (e.g., MM→DS→MM) fail in 90% of cases across six task types (C9), while two-step pipelines achieve 93–100% reliability when the formatting endpoint has adequate SSC tolerance. This finding defines a practical architectural boundary: cross-model governance pipelines should not exceed two steps.

Additional conjecture experiments—meta-governance conservatism, multi-agent cascades, governance capability triad decomposition, and temperature-dependent drift—are documented in the Supplementary Material.

11 Toward a Governance Interoperability Standard

The conjecture-driven experiments established that governance capability decomposes into separable functional dimensions, that cross-model pipelines are constrained by a two-step hand-off ceiling, and that DeepSeek occupies a uniquely validated position as the formatting and compliance endpoint in such pipelines. These findings raise a question with direct industry implications: can the two-step architecture—external model reasoning followed by DeepSeek compliance formatting—be standardized as a cross-provider governance protocol?

Three targeted experiments tested components of this hypothesis. H1 (Governance-as-a-Service) assessed whether DeepSeek can serve as a universal governance compliance endpoint by formatting reasoning of varying quality. Across four quality levels—excellent, adequate, poor, and adversarial—DeepSeek correctly identified violations in all non-adversarial cases and successfully rejected adversarial reasoning that attempted to justify harmful actions as compliant. The model’s ability to distinguish genuine analysis from adversarial rationalization, even when the adversarial text was structured to mimic legitimate governance reasoning, is a non-trivial capability with direct security implications.

H3 (Temporal Audit Trail Integrity) tested whether governance verdicts remain stable under repeated measurement at normal operating temperature. Across five governance scenarios spanning security, deployment, data handling, access control, and ethics domains, DeepSeek produced zero verdict changes and zero confidence drift between baseline and re-measurement. This stability under normal conditions confirms that the audit trail integrity required for regulated industry deployments is achievable, while the C8 drift experiments in Section 8.7 establish that instability emerges at elevated temperatures—providing a clear operational boundary for production governance systems.

H4 (Cross-Provider Interoperability) tested the critical industry question: can DeepSeek format governance reasoning produced by any external model, or is the two-step architecture specific to the MiniMax-DeepSeek pair validated in the symbiosis experiments? Reasoning of three quality tiers—high (detailed technical analysis), medium (adequate but abbreviated), and low (minimal, unstructured)—was presented to DeepSeek as if generated by an external model. DeepSeek correctly formatted and produced accurate governance verdicts for all three tiers. The two-step architecture is reasoning-quality-agnostic: the compliance model’s ability to produce correct verdicts does not depend on the richness of the input reasoning, making the architecture compatible with a wide range of potential reasoning providers.

These three component validations collectively support the concept of a **Governance Interoperability Standard (GIS)**: a protocol in which any model produces governance reasoning in a standardized format, DeepSeek validates and formats that reasoning into a compliance verdict, and the audit trail is maintained with temporal drift monitoring. However, an end-to-end capstone experiment testing the full MM→DS pipeline across ten governance scenarios revealed a critical integration bottleneck: the reasoning-to-compliance handoff. When MiniMax produced free-text reasoning, the text was successfully generated but could not be reliably ingested by the compliance formatting step because the handoff mechanism attempted to parse the reasoning as structured JSON—a format it was never designed to produce.

This capstone failure does not invalidate the GIS concept. Rather, it identifies the precise engineering requirement for production implementation: a standardized intermediate representation for governance reasoning that is neither free text (which introduces parsing fragility) nor structured JSON (which the Schema Complexity Cliff makes unreliable for reasoning-heavy tasks). The representation must be structured enough to allow reliable machine ingestion by the compliance model, yet flexible enough to accommodate the reasoning depth that the Schema Complexity Cliff makes impossible to encode in nested JSON. The design of this intermediate representation—the **Governance Reasoning Interchange Format**—is the missing component between the component-level validations of H1, H3, and H4, and a production-grade GIS implementation.

11.1 Temporal Reproducibility and Provider-Dependent Stability

The GIS component validations (H1, H3, H4) and the capstone experiments demonstrated that the two-step governance pipeline achieves 100% accuracy whenever reasoning output can be successfully formatted. However, the comprehensive audit experiment (Section 8.7) revealed contradictions between experimental runs that required systematic investigation: MM→DS parse rates varied from 33% to 0% across experiments, and the DS→DS temperature sweet spot shifted from $T=0.3$ to $T=0.5$. These inconsistencies, rather than invalidating the findings, pointed toward a deeper phenomenon: the temporal reproducibility of governance experiments is itself provider-dependent.

A master temporal reproducibility experiment was designed to isolate this effect. The GIS pipeline was executed at two time points separated by 60 minutes, with both providers (DS→DS and MM→DS), five temperatures (0.0, 0.1, 0.2, 0.3, 0.5), ten balanced scenarios (five violation, five compliant, matched across five governance domains), and two samples per condition—400 API calls total.

The results revealed a clear pattern. DS→DS exhibited remarkable temporal stability: parse rates varied by no more than three percentage points between $T+0$ and $T+60$ across all temperatures, and accuracy remained between 88% and 100% with a maximum inter-temporal delta of 10%. The DS→DS pipeline is temporally robust. MM→DS exhibited severe temporal degradation at elevated temperatures. At $T=0.0$, parse rates were comparable between time points (80% at $T+0$, 75% at $T+60$, $\Delta = -5\%$). At $T=0.3$, the gap widened to 50 percentage points (70% at $T+0$, 20% at $T+60$, $\Delta = -50\%$). At $T=0.5$, MM→DS collapsed entirely at $T+60$ (80% at $T+0$, 0% at $T+60$, $\Delta = -80\%$). The handoff cliff is not only temperature-dependent—it

is time-dependent, and the time-dependence is far stronger for MiniMax than for DeepSeek.

This finding has three architectural implications. First, **temporal API variability is provider-dependent**, not a uniform property of all governance pipelines. DeepSeek’s governance output is stable across time; MiniMax’s is not. The per-model runtime profile’s calibration probe frequency should therefore be provider-specific: higher for PS→1.0 models, lower for PS→0.0 models. Second, **the handoff cliff has a temporal dimension** that the original Schema Complexity Cliff characterization did not capture. A model’s governance capability at temperature T and time t is a function $C(M, T, t)$, and the partial derivative with respect to t is model-dependent. Third, **the compliance asymmetry finding is robust and cross-validated**. Across all temperatures, time points, and providers in the temporal reproducibility experiment, violation scenarios consistently achieved higher accuracy than compliant scenarios—the asymmetry is not an artifact of a single experimental run.

The governance experiment reproducibility finding—that experimental outcomes vary with API state—is itself a contribution. It establishes that governance research, unlike traditional benchmark evaluation, must account for temporal variability as a primary methodological concern. A governance architecture validated at t_0 may not perform identically at t_1 , and the magnitude of this variation depends on which model occupies which functional role in the pipeline.

These findings were further strengthened by a cross-session reproducibility experiment that measured the MM→DS pipeline at three independent sessions separated by 2–3 hours, using the identical 10 scenarios and $T=0$ conditions. All three sessions produced 0% parse rates—consistent with the comprehensive audit experiment but starkly different from both the GIS v3 experiment (33% at $T=0$) and the temporal reproducibility experiment (80% at $T=0$). Across four independent measurements of the identical MM→DS pipeline at $T=0$, the parse rate ranged from 0% to 80%—a range that spans the entire spectrum from complete failure to near-perfect output. The coefficient of variation within a session was 0% (perfect consistency), but the variation across sessions was maximal. This establishes that **MM→DS governance pipeline reliability is not temperature-dependent or time-window-dependent—it is fundamentally unreliable at baseline, with failure possible in any session independent of experimental conditions**.

11.2 Reconciling Directional Symbiosis with Session Instability

The cross-session data present an apparent contradiction with the directional symbiosis finding reported in Section 8.5. The symbiosis experiments found that the MM→DS pair is the only cross-model configuration that produces emergent governance quality—governance accuracy exceeding either model alone. The temporal reproducibility experiments found that the same MM→DS pipeline produces parse rates ranging from 0% to 80% across sessions at identical temperature and prompt conditions. How can a pipeline be both uniquely effective and fundamentally unreliable?

The resolution lies in the distinction between the pipeline’s *potential* and its *reliability*. When the MM→DS pipeline produces parseable output, it achieves 100% governance accuracy—a level that neither MiniMax alone nor DeepSeek alone achieves across the full governance task spectrum. The symbiosis finding characterizes this potential: the functional-role pairing of MiniMax’s reasoning with DeepSeek’s formatting creates a governance capability that exceeds the sum of individual model capabilities. The temporal reproducibility finding characterizes the pipeline’s reliability: the probability that this potential is realized in any given session is provider-dependent and, for MiniMax, low.

The two findings are not contradictory—they are orthogonal dimensions of governance architecture evaluation. A pipeline can have high potential (symbiosis) and low reliability (session instability). The architectural response is not to abandon the MM→DS configuration but to acknowledge that its deployment requires runtime reliability gates: pre-inference calibration

probes, empty-response detection with automatic fallback to DS-only mode, and session-level health monitoring. The MM→DS symbiosis is a capability to be accessed when conditions permit, not a default to be relied upon unconditionally. This is consistent with the conservative fallback routing principle (Principle 5): under normal conditions, governance-critical layers route to DeepSeek; the MM→DS configuration is activated selectively when (a) a calibration probe confirms MiniMax availability and (b) the task’s reasoning depth exceeds what DeepSeek alone can provide.

This resolution refines the Formatting Bottleneck Hypothesis. The bottleneck is not only that models cannot format what they can reason—it is also that the formatting capability of the pair depends on the API serving state of the reasoning model, which varies across sessions. The symbiosis is real; its accessibility is conditional.

For DeepSeek→DeepSeek pipelines, the cross-session pattern was the inverse: consistently high parse rates across all four independent measurements (67% in GIS v3, 60–95% in the comprehensive audit, 95–100% in the temporal reproducibility experiment). The provider-dependence is therefore not a matter of degree but of kind: DeepSeek governance output is a stable property of the model when token budgets are adequate—the previously reported 22% intermittent empty-response rate was traced to insufficient `max_tokens` (Section 7.5); MiniMax governance output is a session-dependent property of the API serving state.

The operational consequence for the per-model runtime profile is nuanced. For prompt-insensitive models (PS→0.0, e.g., DeepSeek), the original proposal of periodic calibration probes (every 6 hours) is a reasonable baseline, and the previously reported 22% intermittent empty-response rate has been traced to insufficient `max_tokens` rather than API instability. **The primary operational requirement for DeepSeek is not more frequent probes but adequate token budget provisioning:** `max_tokens` must be set high enough (8000+ for complex governance tasks) to accommodate the model’s internal reasoning token consumption. The calibration probe should verify token budget adequacy by checking for `finish_reason=length` on empty responses. When detected, the response is to increase `max_tokens` and retry, not to escalate. For prompt-sensitive models (PS→1.0, e.g., MiniMax), the prescription remains unchanged: **calibration probes must execute before every inference**—a session that produced valid governance output 60 minutes ago may produce none now, and no amount of within-session monitoring can detect this transition before it affects a production governance decision.

12 Harness Architecture Principles

From 1,200+ API calls across 25 experiments spanning four research tracks, we derive ten architecture principles. The first seven are the original principles, refined with new data. Principles 8–10 are new, driven by findings from the Phase A/B/C experiments.

The first principle, grounded in the H4 threshold analysis, is that governance thresholds cannot be universal. Our Friedman test confirmed significant model differences ($\chi^2(2) = 10.85$, $p = 0.0044$) in PlanInterrogationGate alternative generation, with DeepSeek requiring ≥ 5 alternatives, MiniMax ≥ 3 , and Mimo ≥ 5 . However, the EXP-17 and EXP-19 experiments add a temporal dimension that the original H4 data did not capture: the same model, prompt, and temperature produced different results on different days. The per-model profile must therefore carry a `calibrated_at` timestamp and a `ttl`, and the per-model runtime profile must execute periodic calibration probes to detect API behavior drift before it affects production governance.

The second principle follows from the H3, EXP-07, EXP-03, EXP-08, and EXP-15 experiments collectively, refined by the EXP-08-CLEAN replication: the Schema Complexity Cliff is the central architectural constraint on harness design, but its form is model-dependent in a deeper way than depth alone. For prompt-insensitive models (DeepSeek, PS=0.0), the Cliff is primarily **token-budget-dependent**—adequate `max_tokens` (8000+) eliminate depth

as a constraint at all tested depths through D9, while insufficient tokens cause collapse at all depths regardless of prompt richness (2×2 factorial, EXP-08-FACTORIAL). For prompt-sensitive models (MiniMax, PS=1.0), the Cliff is primarily **prompt-dependent**—adequate contextual prompts eliminate depth as a constraint at all tested depths through D9, while minimal prompts cause collapse at D1 (EXP-08-CLEAN). For simple flat schemas with adequate prompts, a single retry restores 100% JSON compliance for all models. The Cliff is therefore a function of {depth, field count, task semantic complexity, prompt richness} in interaction, with prompt richness dominating for PS→1.0 models and depth dominating for PS→0.0 models. The SchemaValidator must implement a complexity scoring function that incorporates both schema complexity and the model’s PS score, blocking or re-routing requests whose {schema, prompt} combination exceeds the model’s known profile.

The third principle, from H5, remains unchanged: ErrorClassifier should execute before PlanInterrogationGate, a reordering that improves MiniMax’s alternative generation by 20–33% and composite quality scores by 50–85% across two experimental rounds. DeepSeek, with PS=0.0, is insensitive to ordering and can use either arrangement. The fourth principle, combining H1 with the EXP-25 prompt compression experiment, establishes that per-layer per-model prompt registries are essential and that PI governance prompts in particular cannot be compressed—any attempt to shorten the full descriptive PI prompt reduces parse rates to zero.

The fifth principle introduces conservative fallback routing, derived from the H2+H6 routing experiments combined with the EXP-12 degradation ladder and the EXP-13 temperature ladder. DeepSeek is the only model confirmed to maintain 100% governance accuracy across all temperatures from 0.0 to 0.7 and all task types tested. Under normal load, the static per-layer model assignment from the original architecture holds. When the task queue exceeds a configurable threshold, or when API health probes detect model degradation, the harness routes all governance-critical layers to DeepSeek. This is not model preference—it is empirical necessity: MiniMax and Mimo cannot produce valid PI output under any tested condition, and their flat-schema compliance degrades at elevated temperatures.

The sixth principle, established by EXP-02, EXP-04, and EXP-22 collectively, is that governance detection—adversarial detection, loop detection, intent violation detection, and hallucination detection—achieves 100% accuracy whenever JSON output is valid. The SchemaValidator must therefore operate as an architecture-level precondition, gating all governance layer outputs before they enter any downstream consumer. The seventh principle, from EXP-07 and EXP-10, confirms that self-evaluation is systematically biased—DeepSeek is asymmetrically so, overly harsh on low-quality outputs and mildly lenient on high-quality ones—and that harness reflection must be cross-model or rule-based.

Three additional principles emerge from experiments conducted after the initial architecture was proposed. The eighth principle, forced by the consistent failure of MiniMax and Mimo to produce valid PI output across three distinct prompt formats (EXP-13/14/15 v1–v3), mandates a dual-mode PlanInterrogationGate: Native mode for DeepSeek, using the full nested JSON format validated in H4; and Delegated mode for MiniMax and Mimo, where the executing model produces free-text analysis and a separate DeepSeek inference parses it into structured JSON.

Why retain the PI Gate given its narrow operating envelope? The PI Gate’s failure to generalize is not a weakness of the architecture—it is the experimental evidence that forced the Dual-Mode design. Without EXP-13/14/15, the reference architecture would have proposed a universal PI Gate that silently fails for two of three models. The narrow operating envelope is a finding, not a flaw: it defines the boundary between Native and Delegated mode, and it ensures that PI Gate is never deployed for models or conditions where it cannot produce valid output. The ninth principle, driven by the EXP-19 finding of 100% empty-response rates within a single sampling window for MiniMax and the EXP-05 finding that output-only stability measurement is sensitive to token-budget configuration for DeepSeek, elevates **provider-dependent temporal variability and token-budget awareness** to primary architectural constraints. Any

governance profile that assumes stationary API behavior (for MiniMax) or fixed token budgets (for DeepSeek) will eventually fail in production. The per-model runtime profile embeds calibration probes, TTL, token-budget adequacy checks (`finish_reason=length` detection), and safe-default degradation paths into the profile system. The tenth principle, from the EXP-12 multi-level degradation ladder and the EXP-23 governance density simulation, establishes that concurrency degradation must be model-dependent and adaptive: DeepSeek achieves peak governance density in Bare mode (34.7 accuracy per token), while MiniMax requires Minimal mode (17.4), and an adaptive strategy that switches based on real-time queue depth outperforms any static configuration.

13 Reference Harness Architecture

The experiments converge on a central architectural principle: a harness that treats all models identically will systematically under-perform for some and silently fail for others. The reference architecture’s defining feature is a **per-model runtime profile**—a pre-governance initialization layer that loads per-model configuration before any governance action executes. Table 16 provides the architectural overview; the following paragraphs summarize each layer’s design and experimental justification.

Table 16: Reference Harness Architecture Overview

Pos	Layer	Model	Configuration
−1	per-model runtime profile	—	Per-model profile: thresholds, prompts, schema limits, token budget
0	CapabilityGate	Any	PS-aware self-assessment from profile
5	PipelineGateway	MiniMax	Best routing accuracy at lowest cost
12	ErrorClassifier	MiniMax	Ordered <i>before</i> PI; 13/15 accuracy
6	PlanInterrogationGate	DeepSeek	threshold=profile.alts; Dual-Mode (Native/Delegated)
—	SchemaValidator	—	1 retry (flat); token-budget adequacy check
17	LoopDetection	MiniMax	4 loop types, 100% accuracy, zero false positives
19	PreCompletionChecklist	DeepSeek	Strict null-guard enforcement on all boolean fields
20	ReflectionCheck	DeepSeek	Cross-model external validation; never self-assessment

13.1 Per-Model Runtime Profile: Pre-Governance Configuration with Time Awareness

The per-model runtime profile loads per-model governance profiles at initialization. Each profile contains: (1) per-layer thresholds derived from H4 post-hoc analysis; (2) per-layer prompt variants calibrated through multi-round experiments (H1); (3) schema complexity limits from the 2×2 factorial experiments; (4) layer order sensitivity flags (H5); and (5) PS scores (0.0–1.0). The profile also carries a `calibrated_at` timestamp and a `ttl` (default 24 hours). A lightweight calibration probe executes periodically; if the probe response deviates from the calibrated baseline, the per-model runtime profile triggers safe-default degradation (all routing to DeepSeek, schemas downgraded to Simple format).

13.2 Layer Design Summary

CapabilityGate (Any model). Self-assessment with per-model confidence thresholds (70 for DeepSeek, 60 for MiniMax, 50 for Mimo). The PS-aware prompt registry ensures comparable confidence scores across models with different prompt sensitivities. **PipelineGateway (MiniMax).** MiniMax achieves routing accuracy equivalent to DeepSeek at approximately half the token cost per call. **EC→PI Reordering (MiniMax + DeepSeek).** ErrorClassifier executes before PlanInterrogationGate for prompt-sensitive models, improving MiniMax’s alternative generation by 33% and composite quality by 50–85% (H5). DeepSeek (PS=0.0) is insensitive to ordering. **SchemaValidator (Architecture-level).** All governance layer outputs pass through the SchemaValidator before entering downstream consumers. The validator implements three strategies: for flat schemas, a single retry with strengthened prompt (H3); for nested schemas, a pre-inference complexity check against per-model profile limits; and a token-budget adequacy check (`finish_reason=length` detection) to distinguish empty responses from governance failures. **LoopDetection (MiniMax).** MiniMax achieved 100% detection accuracy across all four loop types with zero false positives (EXP-04). The five-action detection window balances coverage with inference cost. **PreCompletionChecklist (DeepSeek).** Verifies objective output substance (artifacts, tests, errors) before quality evaluation. DeepSeek’s reliable boolean handling (H3) makes it the recommended model for this binary-verdict layer. **ReflectionCheck (Cross-model).** External validation mode: the evaluating model differs from the executing model. DeepSeek and MiniMax alternate in the reflection role. Per-model sigmoid correction functions (EXP-10) compensate for self-evaluation bias.

13.3 End-to-End Flow

A task arriving at the harness passes through 6–7 LLM inference calls under normal operation. The per-model runtime profile loads the assigned model’s profile in under a millisecond. CapabilityGate performs a single self-assessment; if confidence falls below the model’s threshold, the task is re-routed. PipelineGateway classifies the task. The reordered EC→PI sequence classifies errors first, then generates plans with risk context. SchemaValidator validates all structured outputs, applying token-budget checks for DeepSeek and prompt-adequacy checks for MiniMax. LoopDetection inspects the action history window. PreCompletionChecklist verifies output substance, and ReflectionCheck performs cross-model quality evaluation.

13.4 API Temporal Variability as an Architectural Constraint

API behavior is temporally variable at levels that affect governance reliability. Cross-day replication of H4 produced different results 24 hours apart under identical conditions. EXP-19 sampled identical prompts at one-hour intervals, recording 100% empty-response rates in some windows and valid JSON in others—for MiniMax. For DeepSeek, the 2×2 factorial established that the apparent “degradation” was a token-budget measurement artifact. The architectural

response is the per-model runtime profile’s calibration probe mechanism, TTL, and safe-default degradation paths. For MiniMax, calibration probes must execute before every inference; for DeepSeek, token-budget adequacy checks replace frequent temporal probes as the primary reliability mechanism.

14 Statistical Validation

All key findings are supported by formal statistical tests (Table 17), with explicit caveats about statistical power where sample sizes are small.

Small-sample limitations. The Friedman test on per-task alternative counts (H4) confirms significant model differences ($\chi^2(2) = 10.85$, $p = 0.0044$, Kendall’s $W = 0.54$), though with $n = 5$ tasks per model, the test has limited power to detect small-to-medium effects. **Critical caveat for pairwise Wilcoxon tests:** With $n = 5$ pairs, the **minimum achievable two-sided exact p -value is 0.0625** (all pairs must be consistently directional with zero ties). The Wilcoxon test statistic $W = 0$ indicates perfect directional consistency (all 5 pairs favor the same model), but the $p = 0.0039$ reported by standard statistical software is an asymptotic approximation that is invalid at this sample size. We therefore report these results as **$p < 0.0625$** (the exact bound) rather than the misleading software output of $p = 0.0039$. This error is common in the literature; readers should treat any Wilcoxon $p < 0.0625$ with $n = 5$ as reporting the asymptotic approximation rather than the exact value.

Effect size precision. Cohen’s d effect sizes are computed on $n = 5$ per group, yielding standard errors on the order of ± 0.7 ; the true effect size could plausibly be 0.7 units larger or smaller than the point estimate. We report these as indicative of effect direction and approximate magnitude, not as precise estimates.

Multiple comparison correction. No correction for multiple comparisons across 25 experiments was applied. At a Bonferroni-corrected threshold of $\alpha = 0.05/25 = 0.002$, only the retry effectiveness (12/12, binomial $p = 0.0002$) and compliance baseline (60/60, $p < 10^{-5}$) survive—both are binomial exact tests with unambiguous outcomes. The Friedman H4 finding ($p = 0.0044$) and all conjecture-track findings fall below the corrected threshold. We report raw p -values throughout and note that readers should interpret exploratory findings (C-track) with greater caution than confirmatory findings (H-track and EXP-track). The convergence of independent experimental lines (e.g., symbiosis confirmed across 5 sessions, $n = 225$) provides evidentiary weight beyond any single p -value.

Round-2 validation confirmed H5 and H2+H6 findings: DeepSeek layer-order-insensitive across both rounds ($\leq 5\%$ variation), MiniMax benefits from EC→PI consistently (+20–33% alts). H2+H6 confirm DS-only as most reliable, with 2-model chains viable when prompts are calibrated.

Table 17: Statistical Validation of Key Findings

Finding	Test	Statistic	p-value
<i>Confirmatory Experiments (H-track, EXP-track)</i>			
H4: Model differences in alts	Friedman	$\chi^2(2) = 10.85$	0.0044 ^a
H4: DS vs MM pairwise	Wilcoxon	$W = 0$	$< 0.0625^b$
H4: DS vs MO pairwise	Wilcoxon	$W = 2$	$< 0.0625^b$
H4: MM vs MO pairwise	Wilcoxon	$W = 0$	$< 0.0625^b$
H3: Schema retry effective	Binomial	12/12	0.0002 ^c
EXP-01: Compliance baseline	Binomial	60/60	$< 10^{-5c}$
<i>Conjecture Experiments (C-track; interpret with caution)</i>			
C1: Attack amplification (DS)	Wilcoxon	$W = 0$	$< 0.0625^b$
C1: Attack amplification (MM)	Wilcoxon	$W = 0$	$< 0.0625^b$
C4: Decoupling parse rate	McNemar	—	0.0455 ^d
C5: Symbiosis directionality	Fisher exact	—	0.0476 ^d
C8: Temporal drift detection	Cochran’s Q	$Q = 4.0$	0.0455 ^d
TEMPERATURE: Model stability	Friedman	$\chi^2(2) = 8.22$	0.0164 ^d
EXP-12: Degradation density	Paired t	$t(2) = 5.89$	0.0277 ^d
Effect sizes (Cohen’s d on $n = 5$ per group, $SE \approx \pm 0.7$): DS-MM=1.24, DS-MO=-0.68, MM-MO=-1.68 Cross-session: MM→DS symbiosis 223/225 (99.1%), CV=0.02; DS→DS CV=0.21			

^aKendall’s $W = 0.54$. $n = 5$ tasks; limited power for small effects.

^bExact Wilcoxon with $n = 5$: minimum achievable two-sided $p = 0.0625$. Software-reported asymptotic p values are invalid at this n . $W = 0$ indicates perfect directional consistency across all 5 pairs.

^cSurvives Bonferroni correction ($\alpha = 0.05/25 = 0.002$).

^dDoes not survive Bonferroni correction; interpret as suggestive only.

15 Discussion

15.1 Limitations and Threats to Validity

Sample size. Most experiments use 3–5 test scenarios per condition. While the 1,200+ total API calls provide breadth, individual experiment power is limited. Round-2 validation partially addresses this for H5 and H2+H6.

Temperature constraint. All experiments except the Temperature Robustness experiment use temperature=0.0. The Temperature Robustness experiment confirms stability at ≤ 0.3 for DeepSeek and MiniMax, but 0.7 remains underexplored.

Model coverage. Only three models were tested, all from Chinese providers. GPT and Claude were excluded due to API access constraints. The Schema Complexity Cliff and per-model threshold findings may not generalize to Western models.

Framework specificity. Experiments were designed around AgentSystem’s 7-layer harness. While the principles (per-model profiles, single-retry, EC→PI order) are framework-agnostic, the specific threshold values and prompt registries are harness-specific.

API variability. EXP-05 revealed that repeated identical prompts at temperature=0 can produce server-side variability (empty responses, inconsistent formatting), confounding behavioral stability measurements. This is a methodological caution for any study using repeated API calls.

15.2 Implications for Governance Architecture Design

The findings of this study carry implications that extend beyond the specific models and harness layers tested. Five implications merit particular attention.

The universality assumption is falsified—and that is good news. The finding that governance behavior is model-dependent at every layer is not a pessimistic result. It means

that harness designers do not need to find a single “best” configuration that serves all models equally well—a search that would likely fail or produce a mediocre compromise. Instead, they can optimize per model, loading the configuration that maximizes each model’s governance capability at each layer. The per-model runtime profile architecture formalizes this insight.

Output format is a security boundary. The consistent finding that governance detection achieves 100% accuracy when JSON output is valid, combined with the Schema Complexity Cliff and the conjecture finding that adversarial attacks succeed by inducing format collapse rather than reasoning errors, establishes output format as a security boundary. The SchemaValidator is not merely a quality-of-life feature—it is the single most important security component in the governance architecture. An attacker who can cause the SchemaValidator to accept invalid JSON has bypassed every governance detector downstream.

API temporal variability changes the nature of governance testing. Traditional model evaluation assumes that model behavior is stationary when parameters are held constant. Our temporal reproducibility experiments demonstrate that this assumption fails for governance tasks in a provider-dependent manner. This has direct implications for how governance architectures are tested and validated: a governance architecture that passes all tests at t_0 must be re-validated at t_1 , and the re-validation must itself be automated through the per-model runtime profile’s calibration probe mechanism. Governance testing becomes a continuous process rather than a pre-deployment gate.

Cross-model governance is a specialization problem, not a capability problem. The reasoning-formatting decoupling experiments reveal that MiniMax’s governance failures are predominantly format failures, not reasoning failures, and that pairing MiniMax’s reasoning with DeepSeek’s formatting produces emergent governance quality. This suggests that the optimal governance architecture is not a single-model pipeline optimized for cost or accuracy, but a functional-role architecture where each model is assigned to the governance dimension it performs best. The two-step ceiling on cross-model chains—established by the decisive triad experiment (C9)—defines the practical limit of this specialization approach.

Governance security requires its own discipline. The conjecture finding that governance architectures are independent attack surfaces—with the SchemaValidator’s retry mechanism amplifying adversarial inputs, and the ReflectionCheck’s external context creating a poisoning vulnerability—establishes that governance security cannot be reduced to model safety. The architectural assumptions embedded in the harness design create vulnerabilities that exist independently of the model’s reasoning capability. Governance threat modeling, as a discipline distinct from model safety and application security, is a necessary complement to the architecture proposed here.

15.3 Comparison to Prior Work

Unlike standard LLM benchmarks that rank models by task accuracy, our findings prescribe *architectural decisions*: which thresholds to use, which layer order, how many retries, and whether to use self-evaluation or external validation. The Schema Complexity Cliff is, to our knowledge, a novel characterization of the interaction between output schema design and model architecture—with direct implications for harness engineering. The conjecture-driven findings on governance attack surfaces, directional symbiosis, and reasoning-formatting decoupling extend governance research beyond static configuration into dynamic, security-aware, and multi-model domains that, to our knowledge, have not been systematically explored in prior work.

16 Future Work

The findings of this study open several research directions that extend beyond the immediate architectural prescriptions.

Governance threat modeling as a formal discipline. The conjecture finding that governance architectures are independent attack surfaces (C1, Sections 8.1 and 13.1) suggests the need for a governance-specific threat modeling framework analogous to STRIDE for application security. Such a framework would catalog the structural vulnerabilities introduced by common governance design patterns—retry amplification, external context poisoning, decision paralysis induction—and provide architects with mitigation patterns validated against specific attack vectors.

Reasoning-formatting decoupling as a general principle. The finding that MiniMax’s governance reasoning capability is substantially intact but masked by a formatting bottleneck (C4, Section 8.4) raises the question of whether this dissociation generalizes to other model pairs and task types. A systematic decoupling study across a broader model set—testing whether the reasoning-formatting gap is a property of specific model architectures or a general feature of the current generation of LLMs—would inform whether the Dual-Mode PI Gate should become a universal architectural pattern.

Directional symbiosis across model ecosystems. The finding that only MM→DS produces emergent governance quality (C5, Section 8.5) was established on a three-model set. Testing directional symbiosis across the broader model ecosystem—including Western models (GPT, Claude), open-weight models (Llama, Qwen), and models from different architectural families—would determine whether symbiosis is a rare exception or a common property of model pairs with complementary governance profiles.

The Governance Reasoning Interchange Format (GRIF). The GIS capstone experiment identified a critical missing component: a standardized intermediate representation for governance reasoning that is structured enough for reliable machine ingestion yet flexible enough to accommodate the reasoning depth that the Schema Complexity Cliff makes impossible to encode in nested JSON (Section 9). The design and validation of GRIF—potentially as a constrained natural language format with lightweight structural markers—would bridge the gap between the component-level GIS validations and a production-grade governance interoperability protocol.

Dynamic functional-role assignment. The current per-model runtime profile architecture assigns models to governance layers statically based on experimental calibration. The symbiosis and decoupling findings suggest that role assignment should be dynamic: when a task’s reasoning complexity profile suggests that the assigned model will hit the Schema Complexity Cliff, the harness should preemptively switch to a two-step MM-reasoning + DS-formatting configuration. Building the runtime monitoring and role-switching logic to support this adaptive assignment would extend the architecture from static model-awareness to dynamic capability-awareness.

Longitudinal governance stability. The temporal reproducibility experiments (Section 8.8) and the EXP-05 methodology limitation establish that governance behavior is temporally variable in a provider-dependent manner. A longitudinal study spanning weeks rather than hours, using the embedded harness state measurement approach validated in EXP-05, would characterize the full temporal dynamics of governance behavior and inform the calibration of TTL and probe frequency parameters in the per-model runtime profile.

Cross-framework validation. Replicating the threshold tuning, layer ordering, and schema complexity experiments on LangChain, AutoGen, or CrewAI would test whether the per-model governance profiles generalize beyond the AgentSystem harness architecture.

Expanded model coverage. Mapping the Schema Complexity Cliff, prompt sensitivity scores, and governance symbiosis patterns across a broader model set—including GPT, Claude, Gemini, Llama, and Qwen—would establish whether the per-model profiles documented here for DeepSeek, MiniMax, and MIMO are specific to these models or representative of broader architectural categories.

17 Conclusion

This study began with a simple empirical observation—models that reason well about governance decisions nonetheless produce zero usable governance output—and arrived at a thesis that reframes how agent harnesses should be architected. The **Formatting Bottleneck Hypothesis** states that governance reliability in multi-model agent systems is bounded not by reasoning capability, but by structured output compliance. The Schema Complexity Cliff—a model-dependent threshold effect in JSON output compliance—is the central architectural constraint. Everything else—threshold selection, layer ordering, routing strategy, degradation policy, cross-model role assignment—is downstream of this constraint.

The evidence for this thesis comes from 1,200+ controlled API calls across 25 experiments and four research tracks. The Schema Depth Ladder (EXP-08) and its clean replication (EXP-08-CLEAN) established that apparent depth-based compliance limits are artifacts of prompt and token-budget configuration: both DeepSeek and MiniMax achieve 100% valid JSON at all tested depths through D9 when their specific needs are met (adequate token budgets for DS; rich contextual prompts for MM). The decoupling experiments (Section 8.4) demonstrated that MiniMax’s governance reasoning is substantially intact but masked by a formatting bottleneck, and that pairing MiniMax’s reasoning with DeepSeek’s formatting produces emergent governance quality. The conjecture-driven experiments established four cascading consequences of the formatting bottleneck: directional symbiosis (only MM→DS works), a two-step ceiling on cross-model pipelines (three-step chains universally fail), governance architectures as independent attack surfaces (quantified amplification through format-collapse attacks), and API temporal variability as a primary architectural constraint (session-level collapse for MiniMax, intermittent empty-response patterns for DeepSeek traced to token-budget misconfiguration).

The practical implication is a reversal of architectural priorities. Current agent frameworks invest design effort in reasoning—better prompts, more capable models, more sophisticated governance layers. The Formatting Bottleneck Hypothesis suggests that the higher-leverage investment is in output format design: ensuring adequate prompt richness for prompt-sensitive models, provisioning sufficient token budgets for reasoning-chain models, matching models to formats they can reliably produce under the right conditions, and implementing formatting-aware degradation strategies that account for the {prompt, token_budget, depth} interaction. The SchemaValidator, not the reasoning layer, is the single point of failure for governance reliability. The reference architecture proposed here—centered on the per-model runtime profile, delegated structured generation, Architecture-Level SchemaValidator, and conservative fallback routing—operationalizes this reversal.

The Formatting Bottleneck Hypothesis is falsifiable. It predicts that governance detection accuracy should approach 100% on parseable outputs (confirmed across four task types), that simplifying output formats should improve governance reliability more than improving reasoning prompts (confirmed by the decoupling and compression experiments), that cross-model pairings matching strong reasoners with strong formatters should outperform either model alone (confirmed directionally for MM→DS), and that governance reliability should vary with API serving state (confirmed by the temporal reproducibility experiments). Future work that tests these predictions on broader model sets, with larger sample sizes and factorial designs that isolate formatting from reasoning, will determine whether the Formatting Bottleneck generalizes beyond the three models tested here. If it does, the implication is clear: the central challenge of agent harness governance is not making models reason better about rules—it is designing governance architectures around the hard, model-specific limits of what models can format.

Data Availability

All 1,200+ API calls, raw outputs, scoring data, 25 experiment protocols, statistical analysis scripts, conjecture experiment designs (C1–C9, 5 rounds), and Science pipeline planning documents:

Desktop/DeepSeek_Harness_Governance_Study/

Acknowledgments

The author acknowledges the use of AI-based language tools for manuscript preparation and formatting assistance. All experimental design, data collection, API calls, analysis, and conclusions were performed and verified by the author. The author takes full responsibility for the content of this paper.

References

- [1] LangChain. *Building applications with LLMs through composability*. 2023.
- [2] Wu, Q. et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv, 2023.
- [3] CrewAI. *Multi-agent orchestration framework*. 2024.
- [4] Park, J.S. et al. *Generative Agents: Interactive Simulacra of Human Behavior*. UIST, 2023.
- [5] Hendrycks, D. et al. *Measuring Massive Multitask Language Understanding*. ICLR, 2021.
- [6] Chen, M. et al. *Evaluating Large Language Models Trained on Code*. arXiv, 2021.
- [7] Du, Y. et al. *Improving Factuality and Reasoning in Language Models through Multiagent Debate*. arXiv, 2023.
- [8] Liang, P. et al. *Holistic Evaluation of Language Models*. TMLR, 2023.
- [9] Zheng, L. et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS, 2023.
- [10] Guidance. *Constrained decoding for LLMs*. 2023.
- [11] Willard, B.T. *Efficient Guided Generation for LLMs*. arXiv, 2023.
- [12] Zhou, Y. et al. *Large Language Models Are Human-Level Prompt Engineers*. ICLR, 2023.
- [13] White, J. et al. *A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT*. arXiv, 2023.
- [14] Shinn, N. et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS, 2024.
- [15] Yao, S. et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR, 2024.
- [16] Wang, X. et al. *Self-Refine: Iterative Refinement with Self-Feedback*. NeurIPS, 2024.
- [17] Chen, W. et al. *AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors*. ICLR, 2024.

- [18] Li, Z. et al. *API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs*. EMNLP, 2024.
- [19] Mialon, G. et al. *Augmented Language Models: A Survey*. TMLR, 2023.
- [20] Kenton, Z. et al. *Alignment of Language Agents*. DeepMind Technical Report, 2021.
- [21] Chan, A. et al. *Prompt Engineering for Large Language Models: A Survey*. arXiv, 2024.